

TimeMachine Clock Control Documentation

Tom Smith

Last Updated January 17, 2023

Current Version: 1.1.3

Quick Reference

Source Code Location: <http://studio.woodtv.net/timemachine/source.py>

TMCClocks Source Code Location: <http://studio.woodtv.net/timemachine/TMCClocks.py>

Error Log Location: TimeMachine Clock Control Error Log.log (same directory as .exe)

Installation

There is no install wizard; just unzip the file located at http://studio.woodtv.net/timemachine/tmcc_1_1_3.zip and run the .exe.

Operation

Using the program is very simple. On the left hand side, you can enter a time you'd like the clocks to count down to and click "Start Countdown" to begin the countdown. Note that times must be entered in the format HH:MM:SS (i.e. 09:57:00) and in 24-hour time format. Clicking "Show Real Time" will return the clocks to displaying the current time.

Beneath these are the quick add/subtract buttons; these will immediately add or subtract the corresponding amount of seconds from a countdown or timer when clicked.

On the right hand side you can choose to countdown a specific amount of time rather than to a specified end time (i.e. a clock for forty-five seconds rather than until noon). You can select a premade button or define your own in the custom field.

Beneath that, you can build your own custom buttons that will count down to a specified time. For example, if you've got a show that has multiple hard outs, you can create buttons for those hard outs and save/load them via File Save/Open. You can create four buttons.

Finally, you can use the Color and Brightness settings to adjust the brightness/color of the clocks. These will however reset to default whenever a new countdown is started or the clocks are switched back to real time.

Hidden Features

There are a couple hidden features of the program that can be used:

Quick Exclude (Daybreak Mode)

If you hold the ALT key on the keyboard when starting a countdown or timer, you can quickly exclude it from appearing in the clocks of your choosing. By default, Camera 2 is the only camera that is excluded here. You can adjust the clocks that are excluded by right clicking on them in the Configuration window (File -> Configuration) and choosing either include or exclude from the context window that appears. The thinking here is that if you're in a show where the anchors typically announce the current time in the teases, they can still see that time under one of the cameras while you're counting down to a hard out. *Excluded clocks will still turn red when there is ten seconds left.*

Show Timer Without Starting

In the event that you want to display a timer in the clock but not start it, you can do so by right clicking on any of the timer buttons. You can then start it with a left click. *This only works with timers. It will not work with countdowns.*

Configuration

By selecting Configure from the file menu, you can adjust the IP address of each clock, add new clocks, delete clocks, enable/disable them, adjust their names and addresses, and change the UDP port that commands are sent through.

To add a clock: press add, enter a name for the clock and an IP address in the fields that appear, then click add again.

To delete a clock: click on the clock in the table to highlight it, then click delete. A confirmation window will appear to ask if you're sure.

To change a clock name or IP: double click on the item you want to change. The name of the clock is only for user reference and has no bearing on operation.

To enable/disable a clock: double click on the clock's line in the Enabled column and choose "True" from the dropdown menu to enable it or "False" to disable it.

The default UDP port on the TimeMachine clocks is set to 7372. This can be changed on the clock itself via web browser if necessary. Changing the setting in this program only changes which port the data is being sent to, NOT which ports the clocks are actually using.

Troubleshooting

Double check the IP addresses of the clocks by double pressing the button on the back of them to make the address scroll across the clock and reference those with the ones in the configuration window. If commands still aren't going through, ensure that the computer is on the same network as the clocks and that the UDP port setting matches.

The default settings can be restored by deleting the files "udp_port" and "clocks" from the app directory.

How it Works

The program is sending commands to each of the clocks in turn as a series of bytes using UDP/IP to port 7372. These commands are part of a publicly available API from the manufacturer which is attached to the end of this document. Most of the functions in the source code are simply building these commands by taking user input and converting them into the proper format for sending.

Each clock can be configured to have a different default color, port, etc. by entering the clock's address in a browser. The default password is tmachine.

Change Log

1.0.1

- Adjusted UI to be larger/easier to read/scale with window.
- Added ability to put a timer into the clock but not start it by right clicking the "Custom" button.
- Added the ability to add or subtract time from running countdowns.

1.0.2

- Fixed a bug that caused the add/subtract buttons to add/remove more time than intended.

1.1

- Fixed a bug that would prevent the program from launching if one of the clocks was unreachable.
- Added a five second timeout to all outgoing UDP/IP data.
- Fixed a bug that would cause custom buttons to save/load incorrectly.
- Added the ability to turn clocks back to real time automatically when a countdown or timer ends.

1.1.2

- Rewrite of code base to make code more efficient/legible.
- Addition of TMClocks module to control clocks.
- Added the ability to change clocks to red automatically when there are ten seconds or less remaining on a countdown or timer.
- Added the ability to exclude clocks from a countdown or timer on the fly by holding the ALT key while starting them.
- Added the ability to exclude or include clocks from the ALT + Click option by right clicking them in the configuration menu.
- The last_show file that's generated when the program is closed that loads user settings on launch has been moved to the user's folder on the C:\ drive so that the data will persist correctly across multiple logins.
- Fixed a bug that would cause the program version to be incorrect in the title bar.

1.1.3

- Fixed a bug that would cause countdowns/timers to end prematurely when one had finished but the clock had not yet switched back to real time.
- Removed the test clock from the clocks installed by default.
- Added the ability to report a bug by launching an email composition window via the help menu.

Source Code

```
# -*- coding: utf-8 -*-  
"""
```

```
TimeMachine Clock Controller 1.1.3  
Created on Tue Jan 17 8:00:00 2023
```

```
@author: TOSmith  
"""
```

```
import wx, pickle, webbrowser, logging, keyboard, os, win32api  
from ObjectListView import ObjectListView, ColumnDefn  
import subprocess as sp  
import TMCClocks as TM
```

```
"""Right-click context menu that appears in the config window to exclude or  
include certain clocks when using Alt+Click"""
```

```
class ExclusionMenu(wx.Menu):  
    def __init__(self, parent, name):  
        super(ExclusionMenu, self).__init__()  
        self.parent = parent  
        self.name = name  
        print(self.name)  
        if self.name not in frame.excluded:  
            exclude = wx.MenuItem(self, wx.NewIdRef(count=1), 'Exclude with Alt+Click')  
            self.Append(exclude)  
            self.Bind(wx.EVT_MENU, self.on_exclude)  
        elif self.name in frame.excluded:  
            include = wx.MenuItem(self, wx.NewIdRef(count=1), 'Include with Alt+Click')  
            self.Bind(wx.EVT_MENU, self.on_include)  
            self.Append(include)  
  
    def on_exclude(self, event):  
        frame.excluded.append(self.name)  
  
    def on_include(self, event):  
        frame.excluded.pop(frame.excluded.index(self.name))
```

```
"""Right-click context menu that allows the user to clear custom buttons."""
```

```
class ContextMenu(wx.Menu):  
    def __init__(self, parent, name):  
        super(ContextMenu, self).__init__()  
  
        self.parent = parent  
        self.name = name  
        clear = wx.MenuItem(self, wx.NewIdRef(count=1), 'Clear')  
        self.Append(clear)  
        self.Bind(wx.EVT_MENU, self.on_clear, clear)
```

```

def on_clear(self, event):
    if self.name == "Custom 1":
        frame.custom_1.SetLabel("Custom 1")
        frame.custom_1_assigned = False
        frame.custom_1_type = 0
        frame.custom_1_time = "HH:MM:SS"
    elif self.name == "Custom 2":
        frame.custom_2.SetLabel("Custom 2")
        frame.custom_2_assigned = False
        frame.custom_2_type = 0
        frame.custom_2_time = "HH:MM:SS"
    elif self.name == "Custom 3":
        frame.custom_3.SetLabel("Custom 3")
        frame.custom_3_assigned = False
        frame.custom_3_type = 0
        frame.custom_3_time = "HH:MM:SS"
    elif self.name == "Custom 4":
        frame.custom_4.SetLabel("Custom 4")
        frame.custom_4_assigned = False
        frame.custom_4_type = 0
        frame.custom_3_time = "HH:MM:SS"

    """About window with program name/contact info."""
class AboutFrame(wx.Frame):
    def __init__(self):
        super().__init__(parent=None, title="About")
        self.SetInitialSize((400,150))
        self.font = wx.Font(12, wx.FONTFAMILY_MODERN, 0, 90, underline = False, faceName
        ="Arial Bold")
        self.sizer = wx.FlexGridSizer(4,1,10,10)
        self.panel = wx.Panel(self)
        self.program_name = wx.StaticText(self.panel, label=f"{frame.title} by Tom Smith")
        self.program_name.SetFont(self.font)
        self.phone = wx.StaticText(self.panel, label="(231) 343-9803")
        self.phone.SetFont(self.font)
        self.work_email = wx.StaticText(self.panel, label="thomas.smith@woodtv.com")
        self.work_email.SetFont(self.font)
        self.home_email = wx.StaticText(self.panel, label="tom.a.smith@me.com")
        self.home_email.SetFont(self.font)
        self.sizer.AddMany([(self.program_name),
                            (self.phone),
                            (self.work_email),
                            (self.home_email)])
        self.panel.SetSizer(self.sizer)
        self.sizer.Layout()
        self.Show()

    """Configuration window for adding/removing clocks, changing options, etc."""

```

```

class ConfigFrame(wx.Frame):
    def __init__(self):
        super().__init__(parent=None, title="Configure")
        self.SetMinSize((300,350))
        self.SetInitialSize((300,350))
        self.panel = wx.Panel(self)
        self.main_sizer = wx.BoxSizer(wx.VERTICAL)
        self.buttons_sizer = wx.BoxSizer(wx.HORIZONTAL)
        self.add_sizer = wx.FlexGridSizer(3,2,10,10)
        self.show_add = False
        self.UDP_sizer = wx.BoxSizer(wx.HORIZONTAL)
        self.UDP_label = wx.StaticText(self.panel, label="UDP Port:")
        self.UDP_field = wx.TextCtrl(self.panel, value=str(frame.UDP_PORT))
        self.UDP_sizer.Add(self.UDP_label, 0, wx.ALL | wx.EXPAND, 5)
        self.UDP_sizer.Add(self.UDP_field, 0, wx.ALL | wx.EXPAND, 5)

        self.list_ctrl = ObjectListView(self.panel, wx.ID_ANY, style=wx.LC_REPORT |
wx.SUNKEN_BORDER | wx.EXPAND)
        self.list_ctrl.cellEditMode = ObjectListView.CELLEDIT_DOUBLECLICK
        self.list_ctrl.Bind(wx.EVT_LIST_ITEM_RIGHT_CLICK, self.on_right_click)
        self.set_columns()
        self.main_sizer.Add(self.list_ctrl, 0, wx.ALL | wx.EXPAND, 5)

        self.auto_change_sizer = wx.FlexGridSizer(2,2,0,0)
        self.auto_change_check = wx.CheckBox(self.panel, label="Auto Real Time")
        self.auto_change_check.SetValue(frame.timer_enabled)
        self.auto_change_check.Bind(wx.EVT_CHECKBOX, self.on_auto_check)
        self.seconds_box = wx.TextCtrl(self.panel, value=str(frame.reset_delay))
        self.auto_red_check = wx.CheckBox(self.panel, label="Turn Red with 10 Seconds Left")
        self.auto_red_check.Bind(wx.EVT_CHECKBOX, self.on_red_check)
        self.auto_red_check.SetValue(frame.red_enabled)
        self.auto_change_sizer.AddMany([(self.auto_change_check),
                                         (self.seconds_box),
                                         (self.auto_red_check)])

        self.add_btn = wx.Button(self.panel, label="Add")
        self.add_btn.Bind(wx.EVT_BUTTON, self.on_add)
        self.delete_btn = wx.Button(self.panel, label="Delete")
        self.delete_btn.Bind(wx.EVT_BUTTON, self.on_delete)
        self.buttons_sizer.AddMany([(self.add_btn),
                                     (self.delete_btn)])

        self.clock_name_label = wx.StaticText(self.panel, label="Clock name:")
        self.clock_name_field = wx.TextCtrl(self.panel)
        self.clock_ip_label = wx.StaticText(self.panel, label="Clock IP:")
        self.clock_ip_field = wx.TextCtrl(self.panel)
        self.add_sizer.AddMany([(self.clock_name_label),
                                (self.clock_name_field),
                                (self.clock_ip_label),

```

```

        (self.clock_ip_field)])
self.add_sizer.ShowItems(show=False)

self.main_sizer.AddMany([(self.auto_change_sizer),
                        (self.UDP_sizer),
                        (self.add_sizer),
                        (self.buttons_sizer)])
self.panel.SetSizer(self.main_sizer)
self.Bind(wx.EVT_CLOSE, self.on_close)
self.Show()

#Toggles value of the timer bool when box is clicked
def on_auto_check(self, event):
    change = self.auto_change_check.GetValue()
    if change == True:
        frame.timer_enabled = True
    elif change == False:
        frame.timer_enabled = False

#Toggles value of the red :10 bool when box is clicked
def on_red_check(self, event):
    change = self.auto_red_check.GetValue()
    if change == True:
        frame.red_enabled = True
    elif change == False:
        frame.red_enabled = False

#Shows add fields and adds the clock then hides the fields
def on_add(self, event):
    if self.show_add == False:
        self.show_add = True
        self.add_sizer.ShowItems(show=True)
        self.main_sizer.Layout()
    elif self.show_add == True:
        if len(self.clock_name_field.GetValue()) > 0 and len(self.clock_ip_field.GetValue()) > 0:
            clock_name = self.clock_name_field.GetValue()
            clock_ip = self.clock_ip_field.GetValue()
            frame.clock_objs.append(TM.clock(clock_name, clock_ip, frame.UDP_PORT, True,
None))
            self.set_columns()
            self.show_add = False
            self.add_sizer.ShowItems(show=False)
            self.clock_name_field.SetValue("")
            self.clock_ip_field.SetValue("")
            self.main_sizer.Layout()
            self.main_sizer.Layout()

#Deletes highlighted clock when button is clicked
def on_delete(self, event):

```



```

        self.list_item_clicked = self.list_ctrl.GetSelectedObject()
        print(self.list_item_clicked.clock)
        dlg = wx.MessageDialog(None, f'Are you sure you want to delete
{self.list_item_clicked.clock}?', 'Delete this clock?', wx.YES_NO | wx.NO_DEFAULT |
wx.ICON_EXCLAMATION)
        result = dlg.ShowModal()
        if result == wx.ID_YES:
            for clocks in frame.clock_objs:
                if clocks.clock == self.list_item_clicked.clock:
                    index = frame.clock_objs.index(clocks)
                    frame.clock_objs.pop(index)
                    print("Removed clock.")
            self.set_columns()
        else:
            dlg.Destroy()

#Saves settings to files when config window is closed
def on_close(self, event):
    print("Writing settings to file.")
    filename = 'clocks'
    outfile = open(filename, 'wb')
    pickle.dump(frame.clock_objs, outfile)
    outfile.close()
    frame.UDP_PORT = int(self.UDP_field.GetValue())
    filename = 'udp_port'
    outfile = open(filename, 'wb')
    pickle.dump(frame.UDP_PORT, outfile)
    outfile.close()
    if self.seconds_box.GetValue().isdigit():
        frame.reset_delay = int(self.seconds_box.GetValue())
        filename = 'reset_delay'
        outfile = open(filename, 'wb')
        pickle.dump(frame.reset_delay, outfile)
        outfile.close()
    self.Destroy()

#Populates the list
def set_columns(self, data=None):
    self.list_ctrl.SetColumns([
        ColumnDefn("Clock", "left", -1, "clock"),
        ColumnDefn("Address", "left", -1, "ip"),
        ColumnDefn("Enabled", "left", -1, "enabled", minimumWidth=75)
    ])
    self.list_ctrl.SetObjects(frame.clock_objs)

#Handles right-click on list items
def on_right_click(self, event):
    obj = self.list_ctrl.GetSelectedObject()
    name = obj.clock

```

```

self.PopupMenu(ExclusionMenu(self, name))

#Window for controlling clock brightness
class BrightnessFrame(wx.Frame):
    def __init__(self):
        super().__init__(parent=None, title="Brightness")
        self.SetInitialSize((350,100))
        self.SetMaxSize((350,100))
        self.panel = wx.Panel(self)
        self.main_sizer = wx.BoxSizer(wx.VERTICAL)
        self.slider = wx.Slider(self.panel, value=frame.brightness, minValue=0, maxValue=100,
style = wx.SL_HORIZONTAL|wx.SL_LABELS)
        self.slider.Bind(wx.EVT_SLIDER, self.adjust_brightness)
        self.ok_btn = wx.Button(self.panel, label="OK")
        self.ok_btn.Bind(wx.EVT_BUTTON, self.on_ok)
        self.main_sizer.Add(self.slider,1,flag = wx.EXPAND)
        self.main_sizer.Add(self.ok_btn)
        self.main_sizer.Layout()
        self.panel.SetSizer(self.main_sizer)
        self.Show()

    def adjust_brightness(self, event):
        try:
            for clock in frame.clock_objs:
                if clock.enabled == True:
                    obj = event.GetEventObject()
                    val = obj.GetValue()
                    clock.brightness(val)
                    frame.brightness = val
        except Exception as e:
            print(e)
            logging.error(f"Error adjusting brightness: {e}")

#Closes window
def on_ok(self, event):
    self.Destroy()

class Controller(wx.Frame):
    def __init__(self):
        super().__init__(parent=None, title="TimeMachine Clock Controller 1.1.3")
        self.title = "TimeMachine Clock Controller 1.1.3"
        self.SetTitle(self.title)
        self.SetMinSize((700,350))
        self.SetInitialSize((700,350))
        self.Bind(wx.EVT_CLOSE, self.on_close)

        "Real Time Timer"
        self.timer_enabled = True

```

```

self.red_enabled = True
self.reset_delay = 10
self.red_timer = wx.Timer(self, id=1)
self.real_timer = wx.Timer(self, id=2)
self.Bind(wx.EVT_TIMER, self.red_time, id=1)
self.Bind(wx.EVT_TIMER, self.real_time, id=2)

'''UDP Port'''
self.UDP_PORT = 7372

'''Menu and Statusbar'''
self.create_menu()
self.statusbar = self.CreateStatusBar(1)
self.statusbar.SetStatusText('Ready.')

self.brightness = 50 #Default brightness value for window at launch
self.last_opened = None
logging.basicConfig(filename='TimeMachine Clock Controller Error Log.log', format='%
(asctime)s %(message)s', datefmt='%m/%d/%Y %l:%M:%S %p', level=logging.DEBUG)

'''Clocks'''
self.clock_objs = [TM.clock("Camera 1", "10.10.200.201", 7372, True, None),
                    TM.clock("Camera 2", "10.10.200.202", 7372, True, None),
                    TM.clock("Camera 3", "10.10.200.203", 7372, True, None),
                    TM.clock("Camera 4", "10.10.200.204", 7372, True, None),
                    TM.clock("Camera 5", "10.10.200.205", 7372, True, None),
                    TM.clock("Camera 6", "10.10.200.206", 7372, True, None),
                    TM.clock("Camera 8", "10.10.200.208", 7372, True, None),
                    TM.clock("CTLA", "10.10.200.210", 7372, True, None)]

self.excluded = [] #Clocks to be ignored when Alt+Click to start a countdown

'''Exclude cam 2 by default.'''
for clock in self.clock_objs:
    if clock.clock == "Camera 2":
        self.excluded.append(clock.clock)

'''UI'''
self.panel = wx.Panel(self)

self.font = wx.Font(12, wx.FONTFAMILY_MODERN, 0, 90, underline = False, faceName
="Arial Bold")
self.wrapper = wx.BoxSizer(wx.HORIZONTAL)

#Countdown Section
self.countdown_sizer = wx.FlexGridSizer(3,1,10,10)
self.countdown_sizer.AddGrowableRow(0,1)
self.countdown_sizer.AddGrowableRow(1,1)
self.countdown_sizer.AddGrowableCol(0,1)

```

```

self.countdown_buttons_sizer = wx.BoxSizer(wx.HORIZONTAL)
self.countdown_field_sizer = wx.BoxSizer(wx.VERTICAL)

self.real_time_button = wx.Button(self.panel, label="Show Real Time")
self.real_time_button.Bind(wx.EVT_BUTTON, self.real_time)
self.start_countdown_button = wx.Button(self.panel, label="Start Countdown")
self.start_countdown_button.Bind(wx.EVT_BUTTON, self.countdown)

self.countdown_field = wx.TextCtrl(self.panel, style=wx.TE_PROCESS_ENTER,
value="HH:MM:SS", name="Countdown")
self.countdown_field.Bind(wx.EVT_TEXT_ENTER, self.on_enter)

self.countdown_buttons_sizer.AddMany([(self.real_time_button, 1, wx.ALL | wx.EXPAND),
                                       (self.start_countdown_button, 1, wx.ALL | wx.EXPAND)])

self.countdown_field_sizer.Add(self.countdown_field, 1, wx.EXPAND)

self.countdown_sizer.AddMany([(self.countdown_buttons_sizer, 1, wx.ALL | wx.EXPAND),
                               (self.countdown_field_sizer, 1, wx.ALL | wx.EXPAND)])

#Quick Math Section
self.quick_math_sizer = wx.GridSizer(4,2,10,10)
self.quick_math_sizer.AddGrowableRow(0,1)
self.quick_math_sizer.AddGrowableRow(1,1)
self.quick_math_sizer.AddGrowableRow(2,1)
self.quick_math_sizer.AddGrowableRow(3,1)
self.quick_math_sizer.AddGrowableCol(0,1)
self.quick_math_sizer.AddGrowableCol(1,1)

self.add_1 = wx.Button(self.panel, label="+1")
self.add_1.Bind(wx.EVT_BUTTON, lambda event: self.adjust(event, 1))
self.add_10 = wx.Button(self.panel, label="+10")
self.add_10.Bind(wx.EVT_BUTTON, lambda event: self.adjust(event, 10))
self.add_30 = wx.Button(self.panel, label="+30")
self.add_30.Bind(wx.EVT_BUTTON, lambda event: self.adjust(event, 30))
self.add_60 = wx.Button(self.panel, label="+60")
self.add_60.Bind(wx.EVT_BUTTON, lambda event: self.adjust(event, 60))
self.minus_1 = wx.Button(self.panel, label="-1")
self.minus_1.Bind(wx.EVT_BUTTON, lambda event: self.adjust(event, -1))
self.minus_10 = wx.Button(self.panel, label="-10")
self.minus_10.Bind(wx.EVT_BUTTON, lambda event: self.adjust(event, -10))
self.minus_30 = wx.Button(self.panel, label="-30")
self.minus_30.Bind(wx.EVT_BUTTON, lambda event: self.adjust(event, -30))
self.minus_60 = wx.Button(self.panel, label="-60")
self.minus_60.Bind(wx.EVT_BUTTON, lambda event: self.adjust(event, -60))

self.quick_math_sizer.AddMany([(self.add_1, 1, wx.ALL | wx.EXPAND),

```

```

        (self.minus_1, 1, wx.ALL | wx.EXPAND),
        (self.add_10, 1, wx.ALL | wx.EXPAND),
        (self.minus_10, 1, wx.ALL | wx.EXPAND),
        (self.add_30, 1, wx.ALL | wx.EXPAND),
        (self.minus_30, 1, wx.ALL | wx.EXPAND),
        (self.add_60, 1, wx.ALL | wx.EXPAND),
        (self.minus_60, 1, wx.ALL | wx.EXPAND)])

self.countdown_field_sizer.Add(self.quick_math_sizer, 1, wx.ALL | wx.EXPAND)

self.wrapper.Add(self.countdown_sizer, 1, flag = wx.ALL | wx.EXPAND, border = 15)

#Timer Section
self.premade_timers_sizer = wx.FlexGridSizer(6,2,10,10)
self.premade_timers_sizer.AddGrowableRow(0,1)
self.premade_timers_sizer.AddGrowableRow(1,1)
self.premade_timers_sizer.AddGrowableRow(2,1)
self.premade_timers_sizer.AddGrowableRow(3,1)
self.premade_timers_sizer.AddGrowableRow(4,1)
self.premade_timers_sizer.AddGrowableRow(5,1)
self.premade_timers_sizer.AddGrowableCol(0,1)
self.premade_timers_sizer.AddGrowableCol(1,1)
self.sixty_seconds = wx.Button(self.panel, label="60 Seconds")
self.sixty_seconds.Bind(wx.EVT_BUTTON, lambda event: self.timer(event, 60))
self.thirty_seconds = wx.Button(self.panel, label="30 Seconds")
self.thirty_seconds.Bind(wx.EVT_BUTTON, lambda event: self.timer(event, 30))
self.ten_seconds = wx.Button(self.panel, label="10 Seconds")
self.ten_seconds.Bind(wx.EVT_BUTTON, lambda event: self.timer(event, 10))
self.five_seconds = wx.Button(self.panel, label="5 Seconds")
self.five_seconds.Bind(wx.EVT_BUTTON, lambda event: self.timer(event, 5))

self.custom_timer_button = wx.Button(self.panel, label="Custom")
self.custom_timer_button.Bind(wx.EVT_BUTTON, lambda event: self.timer(event,
self.custom_timer_field.GetValue()))
self.custom_timer_button.Bind(wx.EVT_RIGHT_DOWN, lambda event:
self.load_timer(event, self.custom_timer_field.GetValue()))
self.custom_timer_field = wx.TextCtrl(self.panel, style=wx.TE_PROCESS_ENTER,
value=":45", name="Timer")
self.custom_timer_field.Bind(wx.EVT_TEXT_ENTER, self.on_enter)

self.custom_1 = wx.Button(self.panel, label="Custom 1", name="Custom 1")
self.custom_1.Bind(wx.EVT_BUTTON, lambda event: self.on_custom(event, 1))
self.custom_1.Bind(wx.EVT_RIGHT_DOWN, self.on_custom_right_click)
self.custom_1_assigned = False
self.custom_1_type = 0
self.custom_1_time = "HH:MM:SS"

self.custom_2 = wx.Button(self.panel, label="Custom 2", name="Custom 2")

```

```

self.custom_2.Bind(wx.EVT_BUTTON, lambda event: self.on_custom(event, 2))
self.custom_2.Bind(wx.EVT_RIGHT_DOWN, self.on_custom_right_click)
self.custom_2_assigned = False
self.custom_2_type = 0
self.custom_2_time = "HH:MM:SS"

```

```

self.custom_3 = wx.Button(self.panel, label="Custom 3", name="Custom 3")
self.custom_3.Bind(wx.EVT_BUTTON, lambda event: self.on_custom(event, 3))
self.custom_3.Bind(wx.EVT_RIGHT_DOWN, self.on_custom_right_click)
self.custom_3_assigned = False
self.custom_3_type = 0
self.custom_3_time = "HH:MM:SS"

```

```

self.custom_4 = wx.Button(self.panel, label="Custom 4", name="Custom 4")
self.custom_4.Bind(wx.EVT_BUTTON, lambda event: self.on_custom(event, 4))
self.custom_4.Bind(wx.EVT_RIGHT_DOWN, self.on_custom_right_click)
self.custom_4_assigned = False
self.custom_4_type = 0
self.custom_4_time = "HH:MM:SS"

```

#Customization Buttons

```

self.color_btn = wx.Button(self.panel, label="Color")
self.color_btn.Bind(wx.EVT_BUTTON, self.get_color)
self.brightness_btn = wx.Button(self.panel, label="Brightness")
self.brightness_btn.Bind(wx.EVT_BUTTON, self.on_brightness)

```

```

self.premade_timers_sizer.AddMany([(self.sixty_seconds, 1, wx.ALL | wx.EXPAND),
                                   (self.thirty_seconds, 1, wx.ALL | wx.EXPAND),
                                   (self.ten_seconds, 1, wx.ALL | wx.EXPAND),
                                   (self.five_seconds, 1, wx.ALL | wx.EXPAND),
                                   (self.custom_timer_button, 1, wx.ALL | wx.EXPAND),
                                   (self.custom_timer_field, 1, wx.ALL | wx.EXPAND),
                                   (self.custom_1, 1, wx.ALL | wx.EXPAND),
                                   (self.custom_2, 1, wx.ALL | wx.EXPAND),
                                   (self.custom_3, 1, wx.ALL | wx.EXPAND),
                                   (self.custom_4, 1, wx.ALL | wx.EXPAND),
                                   (self.color_btn, 1, wx.ALL | wx.EXPAND),
                                   (self.brightness_btn, 1, wx.ALL | wx.EXPAND)])

```

```

self.wrapper.Add(self.premade_timers_sizer, 1, flag = wx.ALL | wx.EXPAND, border = 15)

```

```

self.widgets = [self.real_time_button,
                self.start_countdown_button,
                self.countdown_field,
                self.add_1,
                self.add_10,
                self.add_30,

```

```

        self.add_60,
        self.minus_1,
        self.minus_10,
        self.minus_30,
        self.minus_60,
        self.sixty_seconds,
        self.thirty_seconds,
        self.ten_seconds,
        self.five_seconds,
        self.custom_timer_button,
        self.custom_timer_field,
        self.custom_1,
        self.custom_2,
        self.custom_3,
        self.custom_4,
        self.color_btn,
        self.brightness_btn]

    for widgets in self.widgets:
        widgets.SetFont(self.font)

    self.panel.SetSizer(self.wrapper)
    self.Layout()
    self.Show()
    self.on_launch(wx.Event)

    """Functions for control buttons."""
    #Returns a total number of seconds as milliseconds
    def duration(self, time):
        time = int((time-10)*1000)
        return time

    #Runs timer
    def on_timer(self, event):
        timer = event.GetEventObject()
        timer = timer.GetName()
        print(timer)
        if timer == 1:
            self.red_time(wx.Event)
        elif timer == 2:
            self.real_time(wx.Event)

    #Starts internal timers for automatic changing to red/real time
    def wait(self, event, duration):
        #Return the color to cyan first
        b1, b2, b3 = int("00", 16), int("FF", 16), int("FF", 16) #Cyan
        self.set_color(b1, b2, b3)

        timer_status = self.red_timer.IsRunning()

```

```

if duration <= 10:
    wait_time = 0
else:
    wait_time = self.duration(duration)

if self.timer_enabled == True and timer_status == True:
    self.red_timer.Stop()
    self.real_timer.Stop()
    self.red_timer.Start(milliseconds=wait_time, oneShot=True)
    self.real_timer.Start(milliseconds=(wait_time+(10*1000))+(self.reset_delay*1000),
oneShot=True)
elif self.timer_enabled == True and timer_status == False:
    self.red_timer.Start(milliseconds=wait_time, oneShot=True)
    self.real_timer.Start(milliseconds=(wait_time+(10*1000))+(self.reset_delay*1000),
oneShot=True)

#Makes clocks red when the internal timer expires (:10 left)
def red_time(self, event):
    if self.red_enabled == True:
        for clock in self.clock_objs:
            if clock.enabled == True:
                clock.get_info()
                if clock.trt != 0:
                    b1, b2, b3 = int("FF", 16), int("00", 16), int("00", 16) #Red
                    self.statusbar.SetStatusText(f"Turning {clock.clock} red.")
                    self.set_color(b1, b2, b3)
        self.statusbar.SetStatusText("Ready.")

#Starts a countdown on all enabled clocks
def countdown(self, event):
    try:
        for clock in self.clock_objs:
            if clock.enabled == True:
                if keyboard.is_pressed("alt"):
                    if clock.clock in self.excluded:
                        continue
                    self.statusbar.SetStatusText(f"Starting countdown on {clock.clock}.")
                    clock.countdown(self.countdown_field.GetValue())
                    clock.get_info()
                    self.wait_time = clock.trt
                    self.wait(wx.Event, self.wait_time)
                    self.statusbar.SetStatusText("Ready.")
    except Exception as e:
        print(e)
        logging.error(f"Error starting countdown: {e}")
        self.statusbar.SetStatusText(f"Error starting countdown on {clock.clock}. Check log.")

#Switches all enabled clocks to real time

```



```

def real_time(self, event):
    timer_status = self.red_timer.IsRunning()
    if timer_status == True:
        self.red_timer.Stop()
    try:
        for clock in self.clock_objs:
            if clock.enabled == True:
                self.statusbar.SetStatusText(f"Setting {clock.clock} to real time.")
                clock.real_time()
            self.statusbar.SetStatusText("Ready.")
    except Exception as e:
        print(e)
        logging.error(f"Error setting real time: {e}")
        self.statusbar.SetStatusText(f"Error setting real time on {clock.clock}. Check log.")

#Starts a timer on all enabled clocks
def timer(self, event, duration):
    try:
        for clock in self.clock_objs:
            if clock.enabled == True:
                if keyboard.is_pressed("alt"):
                    if clock.clock in self.excluded:
                        continue
                    self.statusbar.SetStatusText(f"Starting timer on {clock.clock}.")
                    clock.timer(duration, False)
                    clock.get_info()
                    self.wait_time = clock.trt
                self.statusbar.SetStatusText("Ready.")
                self.wait(wx.Event, self.wait_time)

    except Exception as e:
        print(e)
        logging.error(f"Error starting timer: {e}")
        self.statusbar.SetStatusText(f"Error starting timer on {clock.clock}. Check log.")

#Adds or removes time from all enabled clocks
def adjust(self, event, adjustment):
    try:
        for clock in self.clock_objs:
            if clock.enabled == True:
                if keyboard.is_pressed("alt"):
                    if clock.clock in self.excluded:
                        continue
                    self.statusbar.SetStatusText(f"Adjusting {clock.clock} by {adjustment} seconds.")
                    clock.adjust(adjustment)
                    clock.get_info()
                    self.wait_time = clock.trt
                self.statusbar.SetStatusText("Ready.")
                self.wait(wx.Event, self.wait_time)

```

```

except Exception as e:
    print(e)
    logging.error(f"Error sending adjustment: {e}")
    self.statusbar.SetStatusText(f"Error sending adjustment to {clock.clock}. Check log.")

#If a custom button is assigned, it runs. Else, it builds the button.
def on_custom(self, event, button):
    if button == 1 and self.custom_1_assigned == True:
        if self.custom_1_type == 1:
            self.countdown_field.SetValue(self.custom_1_time)
            self.countdown(wx.EVT_BUTTON)
        elif self.custom_1_type == 2:
            self.timer(wx.Event, self.custom_1_time)
    elif button == 2 and self.custom_2_assigned == True:
        if self.custom_2_type == 1:
            self.countdown_field.SetValue(self.custom_2_time)
            self.countdown(wx.EVT_BUTTON)
        elif self.custom_2_type == 2:
            self.timer(wx.Event, self.custom_2_time)
    elif button == 3 and self.custom_3_assigned == True:
        if self.custom_3_type == 1:
            self.countdown_field.SetValue(self.custom_3_time)
            self.countdown(wx.EVT_BUTTON)
        elif self.custom_3_type == 2:
            self.timer(wx.Event, self.custom_3_time)
    elif button == 4 and self.custom_4_assigned == True:
        if self.custom_4_type == 1:
            self.countdown_field.SetValue(self.custom_4_time)
            self.countdown(wx.EVT_BUTTON)
        elif self.custom_4_type == 2:
            self.timer(wx.Event, self.custom_4_time)
    else:
        message = "Clock Type"
        caption = "Choose a clock type:"
        choices = ["Countdown", "Timer"]
        parent = None
        dlg = wx.SingleChoiceDialog(parent, message, caption, choices, style=wx.OK |
wx.CANCEL | wx.CENTRE, pos=wx.DefaultPosition)
        #dlg.ShowModal()
        if dlg.ShowModal() == wx.ID_OK:
            choice = dlg.GetStringSelection()
            if choice == "Countdown":
                choice = 1
            elif choice == "Timer":
                choice = 2
            dlg = wx.TextEntryDialog(self, 'Name the button:', 'Custom Button')
            if dlg.ShowModal() == wx.ID_OK:
                if button == 1:
                    self.custom_1.SetLabel(dlg.GetValue())

```

```

        self.custom_1_assigned = True
        self.custom_1_type = choice
    elif button == 2:
        self.custom_2.SetLabel(dlg.GetValue())
        self.custom_2_assigned = True
        self.custom_2_type = choice
    elif button == 3:
        self.custom_3.SetLabel(dlg.GetValue())
        self.custom_3_assigned = True
        self.custom_3_type = choice
    elif button == 4:
        self.custom_4.SetLabel(dlg.GetValue())
        self.custom_4_assigned = True
        self.custom_4_type = choice
    dlg = wx.TextEntryDialog(self, 'Set a time (HH:MM:SS):', 'Custom Button')
    if dlg.ShowModal() == wx.ID_OK:
        if button == 1:
            self.custom_1_time = dlg.GetValue()
        elif button == 2:
            self.custom_2_time = dlg.GetValue()
        elif button == 3:
            self.custom_3_time = dlg.GetValue()
        elif button == 4:
            self.custom_4_time = dlg.GetValue()

```

#Launches a color picker and passes the hex to set_color

```

def get_color(self, event):
    dlg = wx.ColourDialog(self)
    dlg.GetColourData().SetChooseFull(True)
    if dlg.ShowModal() == wx.ID_OK:
        new_color = dlg.GetColourData().Colour
        new_color_string = new_color.GetAsString(flags=wx.C2S_HTML_SYNTAX)
        byte_1 = int(new_color_string[1:3], 16)
        byte_2 = int(new_color_string[3:5], 16)
        byte_3 = int(new_color_string[5:7], 16)
        self.set_color(byte_1, byte_2, byte_3)

```

#Sets the color of enabled clocks

```

def set_color(self, byte_1, byte_2, byte_3):
    try:
        for clock in self.clock_objs:
            if clock.enabled == True:
                self.statusbar.SetStatusText(f"Changing color on {clock.clock}.")
                clock.color(byte_1, byte_2, byte_3)
                self.statusbar.SetStatusText("Ready.")
    except Exception as e:
        print(e)
        logging.error(f"Error changing color: {e}")
        self.statusbar.SetStatusText(f"Error changing color on {clock.clock}. Check log.")

```

```

#Launches brightness controls
def on_brightness(self, event):
    BrightnessFrame()

"""Functions for building the menu/executing options."""

#Creates menu bar & associated options
def create_menu(self):
    menu_bar = wx.MenuBar()

    file_menu = wx.Menu()
    new_option = file_menu.Append(wx.ID_ANY, 'New', 'Reset the custom buttons and build a
new show.')
    self.Bind(event=wx.EVT_MENU,handler=self.on_new_option,source=new_option)

    open_option = file_menu.Append(wx.ID_ANY, 'Open', 'Open a custom button file.')
    self.Bind(event=wx.EVT_MENU,handler=self.on_open_option,source=open_option)

    save_option = file_menu.Append(wx.ID_ANY, 'Save', 'Save a custom button file.')
    self.Bind(event=wx.EVT_MENU,handler=self.on_save_option,source=save_option)

    configure_option = file_menu.Append(wx.ID_ANY, 'Configure', 'Configure clocks.')

self.Bind(event=wx.EVT_MENU,handler=self.on_configure_option,source=configure_option)
    menu_bar.Append(file_menu, '&File')

    help_menu = wx.Menu()
    about_option = help_menu.Append(wx.ID_ANY, 'About','Information about this program.')
    self.Bind(event=wx.EVT_MENU,handler=self.on_about_option,source=about_option)

    documentation_option = help_menu.Append(wx.ID_ANY, 'Documentation',"Opens a PDF of
this program's documentation.")

self.Bind(event=wx.EVT_MENU,handler=self.on_documentation_option,source=documentation
_option)

    error_log_option = help_menu.Append(wx.ID_ANY, "Error Log","Opens the error log.")

self.Bind(event=wx.EVT_MENU,handler=self.on_error_log_option,source=error_log_option)

    bug_report_option = help_menu.Append(wx.ID_ANY, "Bug Report","Report a bug via
email.")

self.Bind(event=wx.EVT_MENU,handler=self.on_bug_report_option,source=bug_report_option)

    menu_bar.Append(help_menu, '&Help')

```

```

self.SetMenuBar(menu_bar)

#Clears all customization
def on_new_option(self, event):
    self.last_opened = None
    self.custom_1.SetLabel("Custom 1")
    self.custom_1_assigned = False
    self.custom_1_type = 0
    self.custom_1_time = "HH:MM:SS"

    self.custom_2.SetLabel("Custom 2")
    self.custom_2_assigned = False
    self.custom_2_type = 0
    self.custom_2_time = "HH:MM:SS"

    self.custom_3.SetLabel("Custom 3")
    self.custom_3_assigned = False
    self.custom_3_type = 0
    self.custom_3_time = "HH:MM:SS"

    self.custom_4.SetLabel("Custom 4")
    self.custom_4_assigned = False
    self.custom_4_type = 0
    self.custom_4_time = "HH:MM:SS"

self.SetTitle(self.title)

#Opens custom button file
def on_open_option(self, event):
    custom_1_name = None
    custom_2_name = None
    custom_3_name = None
    custom_4_name = None
    with wx.FileDialog(self, "Open show file", wildcard="show files (*.show)|*.show",
        style=wx.FD_OPEN | wx.FD_FILE_MUST_EXIST) as fileDialog:

        if fileDialog.ShowModal() == wx.ID_CANCEL:
            return # the user changed their mind

    # Proceed loading the file chosen by the user
    pathname = fileDialog.GetPath()
    self.last_opened = pathname
    try:
        infile = open(pathname, 'rb')
        [custom_1_name, self.custom_1_assigned, self.custom_1_time, self.custom_1_type,
        custom_2_name, self.custom_2_assigned, self.custom_2_time, self.custom_2_type,
        custom_3_name, self.custom_3_assigned, self.custom_3_time, self.custom_3_type,
        custom_4_name, self.custom_4_assigned, self.custom_4_time, self.custom_4_type]
        = pickle.load(infile)

```

```

        self.custom_1.SetLabel(custom_1_name)
        self.custom_2.SetLabel(custom_2_name)
        self.custom_3.SetLabel(custom_3_name)
        self.custom_4.SetLabel(custom_4_name)
        if self.last_opened:
            last_slash = self.last_opened.rfind("\\")
            last_period = self.last_opened.rfind(".")
            file = self.last_opened[last_slash+1:last_period]
            self.SetTitle(f"{self.title} - {file}")
    except Exception as e:
        print(e)
        logging.error(f"Error opening {self.last_opened}: {e}")

#Saves custom button file
def on_save_option(self, event):
    with wx.FileDialog(self, "Save show file", wildcard="show files (*.show)|*.show",
style=wx.FD_SAVE | wx.FD_OVERWRITE_PROMPT) as fileDialog:
        if fileDialog.ShowModal() == wx.ID_CANCEL:
            return # the user changed their mind

    # save the current contents in the file
    pathname = fileDialog.GetPath()
    self.last_opened = pathname
    try:
        print("Writing settings to file.")
        custom_1_name = self.custom_1.GetLabel()
        custom_2_name = self.custom_2.GetLabel()
        custom_3_name = self.custom_3.GetLabel()
        custom_4_name = self.custom_4.GetLabel()
        outfile = open(pathname, 'wb')
        pickle.dump([custom_1_name, self.custom_1_assigned, self.custom_1_time,
self.custom_1_type,
                        custom_2_name, self.custom_2_assigned, self.custom_2_time,
self.custom_2_type,
                        custom_3_name, self.custom_3_assigned, self.custom_3_time,
self.custom_3_type,
                        custom_4_name, self.custom_4_assigned, self.custom_4_time,
self.custom_4_type], outfile)
        outfile.close()
        if self.last_opened:
            last_slash = self.last_opened.rfind("\\")
            last_period = self.last_opened.rfind(".")
            file = self.last_opened[last_slash+1:last_period]
            self.SetTitle(f"{self.title} - {file}")
    except Exception as e:
        print(e)
        logging.error(f"Error saving {self.last_opened}: {e}")

#Launches configuration window

```

```

def on_configure_option(self, event):
    ConfigFrame()

#Launches about window
def on_about_option(self, event):
    AboutFrame()

#Opens documentation in default browser
def on_documentation_option(self, event):
    path = 'http://studio.woodtv.net/timemachine/documentation.pdf'
    webbrowser.open_new(path)

#Opens the error log in notepad
def on_error_log_option(self, event):
    program_name = "notepad.exe"
    file_name = "TimeMachine Clock Controller Error Log.log"
    sp.Popen([program_name, file_name])

#Launches an email composition window addressed to me for users to send in bugs
def on_bug_report_option(self, event):
    win32api.ShellExecute(0,'open',f'mailto:thomas.smith@woodtv.com?subject={self.title} Bug Report',None,None,0)

"""Data persistence functions"""
#Writes information about the current state of the window to file for
#loading next launch
def on_close(self, event):
    print('Closing')
    print("Writing settings to file.")
    home_dir = os.path.expanduser("~")
    filename = f'{home_dir}\\last_show'
    outfile = open(filename, 'wb')
    countdown = self.countdown_field.GetValue()
    timer = self.custom_timer_field.GetValue()
    pickle.dump([countdown, timer, self.last_opened, self.timer_enabled,
                self.reset_delay, self.red_enabled, self.excluded], outfile)
    outfile.close()
    self.Destroy()

#Gets and loads info when program launches
def on_launch(self, event):
    #Load clock info
    try:
        filename = 'clocks'
        infile = open(filename,'rb')
        self.clock_objs = pickle.load(infile)
        infile.close()
    except Exception as e:
        print(f"Error loading clock info: {e}")

```

```

        logging.error(f"Error loading clock info: {e}")

#Load UDP Port info
try:
    filename = 'udp_port'
    infile = open(filename,'rb')
    self.UDP_PORT = pickle.load(infile)
    infile.close()
except Exception as e:
    print(f"Error loading UDP port info: {e}")
    logging.error(f"Error loading UDP port info: {e}")

#Load timer and previous show info
try:
    """We save the file to the user's root directory so that each user
    sees their own buttons and settings on launch"""
    home_dir = os.path.expanduser("~")
    filename = f'{home_dir}\\last_show'
    infile = open(filename,'rb')
    [countdown, timer, self.last_opened, self.timer_enabled,
     self.reset_delay, self.red_enabled, self.excluded] = pickle.load(infile)
    self.countdown_field.SetValue(countdown)
    self.custom_timer_field.SetValue(timer)
    if self.last_opened:
        last_slash = self.last_opened.rfind("\\")
        last_period = self.last_opened.rfind(".")
        file = self.last_opened[last_slash+1:last_period]
        self.SetTitle(f"{self.title} - {file}")
    infile.close()
except Exception as e:
    print(f"Error loading timer and previous show: {e}")
    logging.error(f"Error loading timer and previous show info: {e}")

#Load custom buttons from last show
try:
    filename = self.last_opened
    infile = open(filename,'rb')
    [custom_1_name, self.custom_1_assigned, self.custom_1_time, self.custom_1_type,
     custom_2_name, self.custom_2_assigned, self.custom_2_time, self.custom_2_type,
     custom_3_name, self.custom_3_assigned, self.custom_3_time, self.custom_3_type,
     custom_4_name, self.custom_4_assigned, self.custom_4_time, self.custom_4_type] =
pickle.load(infile)
    self.custom_1.SetLabel(custom_1_name)
    self.custom_2.SetLabel(custom_2_name)
    self.custom_3.SetLabel(custom_3_name)
    self.custom_4.SetLabel(custom_4_name)

except Exception as e:
    print(f"Error loading custom buttons: {e}")

```



```

        logging.error(f"Error loading custom buttons: {e}")

#Test clock connections and get info
try:
    for clock in self.clock_objs:
        if clock.enabled == True:
            clock.get_info()
            if clock.timeout == True:
                clock.enabled = False
                dlg = wx.MessageDialog(None, f"Couldn't connect to {clock.clock}. Please check
its connection for power and network.", 'Error', wx.OK | wx.ICON_ERROR)
                dlg.ShowModal()
except Exception as e:
    print(f"Error getting clock info or connection: {e}")
    logging.error(f"Error getting clock info or connection: {e}")

"""UI Management/Hidden Option Functions"""
"""Additionally, if ALT is held while clicking to start a countdown or timer,
specified clocks will be excluded from that action."""

#Show but not start timer
def load_timer(self, event, duration):
    try:
        for clock in self.clock_objs:
            if clock.enabled == True:
                clock.timer(duration, True)
    except Exception as e:
        print(f"Error loading timer in {clock.clock}: {e}")
        logging.error(f"Error loading timer in {clock.clock}: {e}")

#On enter for TextCtrl
def on_enter(self, event):
    field = event.GetEventObject()
    field = field.GetName()
    print(field)
    if field == "Countdown":
        self.countdown(wx.Event)
    elif field == "Timer":
        self.timer(wx.Event, self.custom_timer_field.GetValue())

#Shows context menu when right clicking on custom buttons, enabling them
#to be cleared
def on_custom_right_click(self, event):
    caller = event.GetEventObject()
    name = caller.GetName()
    print(name)
    self.PopupMenu(ContextMenu(self, name))

if __name__ == "__main__":

```

```
app=[]; app = wx.App(None)
frame = Controller()
app.MainLoop()
```

TMCClocks

```
# -*- coding: utf-8 -*-  
"""
```

TMC Clocks

Created on Fri Jan 6 07:29:34 2023

@author: TOSmith

This module is designed to establish the clock object class and associated methods that allow for control and receiving of information from TimeMachine Corporation POE B series clocks. It uses the manufacturer provided API located at <https://www.timemachinescorp.com/wp-content/uploads/TimeMachinesControlAPI.pdf>

I've built this module so that the code for the UI is more readable, and so that changes can be made more quickly and efficiently if necessary.

```
"""
```

```
from datetime import datetime, timedelta  
import socket  
import struct
```

```
class clock(object):  
    def __init__(self, clock, ip, port, enabled, mode):  
        self.clock = clock  
        self.ip = ip  
        self.UDP_PORT = port  
        self.enabled = True  
        self.mode = None  
        self.device_type = None  
        self.mac = None  
        self.displayed_time = None  
        self.trt = None  
        self.countdown_time = None  
        self.timer_time = None  
        self.timeout = None
```

```
    #Commands from the API
```

```
    self.API = {"UpTimer Mode": 0xA2,  
                "UpTimer Start/Pause": 0xA3,  
                "UpTimer Reset": 0xA4,  
                "DownTimer Mode": 0xA5,  
                "DownTimer Start/Pause": 0xA6,  
                "DownTimer Reset": 0xA7,  
                "Time Mode": 0xA8,  
                "DotMatrix": 0xA9,  
                "UpTimer While Running": 0xAA,  
                "DownTimer While Running": 0xAB,
```

```

        "Countdown to Date Timer Reset": 0xB9,
        "Countdown to Date Start/Pause": 0xBA,
        "Execute Stored Program": 0xB8,
        "Relay Close": 0xB4,
        "Relay Toggle": 0xBC,
        "Revert to Time Timeout": 0xBB,
        "Dimmer Set": 0xB5,
        "Color Set": 0xB6}

```

#Send a command to the clock.

```

def command(self, command):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.settimeout(5)
    sock.sendto(bytes(command), (self.ip, self.UDP_PORT))
    reply = sock.recv(40)
    #print(reply)
    if reply:
        return reply
    sock.close()
    else:
        sock.close()

```

#Get and store the info from the clock.

```

def get_info(self):
    try:
        reply = self.command([0xA1, 0x04, 0xB2])
        key = bytearray(reply)
        device_type = int(key[0])
        if device_type == 1:
            self.device_type = "POE"
        elif device_type == 2:
            self.device_type = "WiFi"
        elif device_type == 3:
            self.device_type = "DotMatrix"
        ip = str(str(key[1])+"."+str(key[2])+"."+str(key[3])+"."+str(key[4]))
        mac = str(str(key[5])+"."+str(key[6])+"."+str(key[7])+"."+str(key[8])+"."+str(key[9])
+"."+str(key[10]))
        firmware = str(str(key[11])+"."+str(key[12]))
        displayed_hours = int(key[15])
        if displayed_hours < 10:
            zero = "0"
            displayed_hours = str(displayed_hours)
            displayed_hours = str(zero+displayed_hours)
        else:
            displayed_hours = str(displayed_hours)
        displayed_minutes = int(key[16])
        if displayed_minutes < 10:
            zero = "0"
            displayed_minutes = str(displayed_minutes)

```

```

        displayed_minutes = str(zero+displayed_minutes)
    else:
        displayed_minutes = str(displayed_minutes)
    displayed_seconds = int(key[17])
    if displayed_seconds < 10:
        zero = "0"
        displayed_seconds = str(displayed_seconds)
        displayed_seconds = str(zero+displayed_seconds)
    else:
        displayed_seconds = str(displayed_seconds)
    self.displayed_time =
str(displayed_hours+":"+displayed_minutes+"."+displayed_seconds)
    self.displayed_time = datetime.strptime(self.displayed_time, "%H:%M:%S")
    trt = self.displayed_time - datetime(1900, 1, 1)
    self.trt = int(trt.total_seconds())
    display_mode = key[19]
    print(f'Device Type: {self.device_type}')
    print(f'IP Address: {ip}')
    print(f'MAC Address: {mac}')
    print(f'Firmware: {firmware}')
    print(f'Displayed time: {displayed_hours}:{displayed_minutes}:{displayed_seconds}')
    if display_mode == 69:
        print('Display mode: Countdown')
        self.mode = "Countdown"
    elif display_mode == 66:
        print('Display mode: Timer')
        self.mode = "Timer"
    elif display_mode == 0:
        print('Display mode: Real Time')
        self.mode = "Real Time"
    self.timeout = False
except socket.timeout as e:
    print(f'Timeout error: {e}')
    self.timeout = True
except Exception as e:
    print(e)

#Change clock color.
def color(self, b1, b2, b3):
    value = [self.API['Color Set'], b1, b2, b3, b1, b2, b3]
    self.command(value)

#Change brightness.
def brightness(self, brightness):
    dimmer_setting = [self.API['Dimmer Set'], brightness, brightness]
    self.command(dimmer_setting)

#Return clock to real time.
def real_time(self):

```

```

    real_time = [0xA8, 0x01, 0x00]
    self.command(real_time)

#Start a countdown.
def countdown(self, time):
    self.countdown_time = datetime.strptime(time, "%H:%M:%S")
    colon_1 = time.index(":")
    colon_2 = time.rfind(":")
    hour = int(time[:colon_1])
    minute = int(time[colon_1+1:colon_2])
    second = int(time[colon_2+1:])
    now = datetime.now()
    month = int(now.strftime("%m"))
    if hour == 0:
        day = int(now.strftime("%d"))+1
    else:
        day = int(now.strftime("%d"))
    year = int(now.strftime("%Y"))
    year = str(struct.pack('<h', year))
    backslash_1 = year.index("\\")
    backslash_2 = year.rindex("\\")
    year_lsb = year[backslash_1+1:backslash_2]
    year_msb = year[backslash_2+1:backslash_2+4]
    year_lsb = "0"+year_lsb
    year_msb = "0"+year_msb
    year_lsb = int(year_lsb, 16)
    year_msb = int(year_msb, 16)
    time = [0xA8, 0x01, 0x00]
    countdown = [0xB9, 0x01, year_lsb, year_msb, month, day, hour, minute, second, 0x1,
0x1]
    start = [0xBA,0x01,0x00]
    self.command(time)
    self.command(countdown)
    self.command(start)

#Starts a timer.
def timer(self, time, hold):
    if time == 60:
        hour = 0
        minute = 1
        second = 0
    else:
        time = str(time)
        colon_count = time.count(":")
        if colon_count == 2:
            colon_1 = time.index(":")
            colon_2 = time.rfind(":")
            hour = int(time[:colon_1])
            minute = int(time[colon_1+1:colon_2])

```

```

        second = int(time[colon_2+1:])
    elif colon_count == 1:
        colon = time.index(":")
        if colon == 0:
            hour = 0
            minute = 0
            second = int(time[colon+1:])
        else:
            hour = 0
            minute = int(time[:colon])
            second = int(time[colon+1:])
    elif colon_count == 0:
        hour = 0
        minute = 0
        second = int(time)
    starting_tenths = 10
    alarm = 1
    alarm_duration = 10
    timer = [self.API['DownTimer Mode'], 0x01, hour, minute, second, starting_tenths, alarm,
alarm_duration]
    timer_start = [self.API['DownTimer Start/Pause'], 0x01, 0x00]
    self.command(timer)
    if hold == False:
        self.command(timer_start)

#Adjust current countdown or timer by up to 60 seconds.
def adjust(self, adjustment):
    self.get_info()
    if self.mode == "Timer":
        current = self.displayed_time
        if adjustment == 60:
            time = current + timedelta(minutes=1)
        elif adjustment == -60:
            time = current + timedelta(minutes=-1)
        else:
            time = current + timedelta(seconds=adjustment)

    time = datetime.strptime(time, "%H:%M:%S")
    colons = time.count(":")
    if colons == 2:
        #Hours
        colon_index = time.find(":")
        second_colon_index = time.rfind(":")
        hours = int(time[:colon_index])
        minutes = int(time[colon_index+1:second_colon_index])
        seconds = int(time[second_colon_index+1:])
    elif colons == 1:
        if len(time) > 3:
            #Minutes

```

```

        colon_index = time.find(":")
        hours = 0
        minutes = int(time[colon_index])
        seconds = int(time[colon_index+1:])
    else:
        #Seconds
        hours = 0
        minutes = 0
        seconds = int(time[1:])

    starting_tenths = 10
    alarm = 1
    alarm_duration = 10
    countdown = [self.API['DownTimer Mode'], 0x01, hours, minutes, seconds,
starting_tenths, alarm, alarm_duration]
    custom_clock_start = [self.API['DownTimer Start/Pause'], 0x01, 0x00]
    self.command(countdown)
    self.command(custom_clock_start)

elif self.mode == "Countdown":
    current = self.countdown_time
    print(current)
    if adjustment == 60:
        new_time = current + timedelta(minutes=1)
    elif adjustment == -60:
        new_time = current + timedelta(minutes=-1)
    else:
        new_time = current + timedelta(seconds=adjustment)
    new_time = datetime.strftime(new_time, "%H:%M:%S")
    print(new_time)
    self.countdown(new_time)

```


TMCClocks Usage

First the module must be imported into the program. Its easiest to import it with an alias, i.e.

```
import TMCClocks as TM
```

From there you can begin building clock objects either as a named variable:

```
clock_1 = TM.clock("Camera 1", "10.10.200.201", 7372, True, None)
```

Or as a list of unnamed variables:

```
self.clock_objs = [TM.clock("Camera 1", "10.10.200.201", 7372, True, None),  
                   TM.clock("Camera 2", "10.10.200.202", 7372, True, None)]
```

Five variables must be established in order when building the object: friendly name, IP address, UDP Port, if the clock is enabled, and the clock's mode.

Friendly name (str): Name that's displayed for the clock in the configuration window

IP Address (str): IP for the clock

UDP Port (int): UDP Port commands should be sent to; default is 7372.

Enabled (bool): Whether or not the clock should be sent commands

Mode (str): Should be established as None type.

From there you can use the built in methods to easily control the clocks. The available methods are:

- command: Sends a command to the clock. Mainly an internal method of the module:
clock.command(command)

- get_info: Gets information from the clock including the clock's model, IP address, MAC address, mode, and currently displayed time. Saves that info so it's accessible from the object:
clock.get_info()

- color: Changes the color of the clock using a color hex code. The three variables that must be passed into it are the three bytes of the hex code:
clock.color(b1, b2, b3)

- brightness: Adjust the brightness of the clock. The variable passed into it is an int representing the brightness percentage (I.E. 50):
clock.brightness(brightness)

- real_time: Returns the clock to displaying the real time:
clock.real_time()

-countdown: Starts a countdown using a string passed as the variable with the format HH:MM:SS:
clock.countdown(time)

-timer: Starts a timer or, if the hold variable is passed as True, puts the timer into the clock but does not start it. The time variable can be either a str or an int:
`clock.timer(time, hold)`

-adjust: Adds or subtracts time from the currently displayed countdown or timer. Adjustment can be passed as a str or int:
`clock.adjust(adjustment)`



Locator Protocol API version 2.1

July 2022

1 Locator Data Query

Product	Firmware Supporting
TM1000	2.6
TM2000	0.3.3
POE Display (B series)	4.9 (unreleased)
POE Display (C series), Supporting 9 digit display	5.2
WiFi Display (B series)	2.6 (unreleased)
WiFi Display (C series)	3.2

*The series letter is determined from the first letter of the serial number of the device

This API was original designed primarily for use by TM-Manager, but is available for management and some remote control of the devices. This later release only supports the mo above. See the 1.1 version of this document for models prior to the version above.

1.1 Device Query

The Locator Data Service is a simple UDP/IP protocol that can be used by other network applications to extract status and location information from TM products.

1.1.1 POE, WiFi, and DotMatrix Device Query Format (Clocks)

Requesting information from the clocks is done by sending a 3 byte message to the Clock, using UDP/IP, to port 7372. The three bytes, in hexadecimal, are: 0xA1 0x04 0xB2 The TM1000/2000's will also respond to a broadcast to the same port.

The response packet is 40 bytes and will be formatted as follows:

Bytes	Description
0	Device Type: 0x01=POE, 0x02=WiFi, 0x03=DotMatrix
1 to 4	client IP address
5 to 10	MAC address
11 and 12	firmware version Major:Minor
13 and 14	NTP Sync Count
15 to 17	Displayed Time / Timer Value: HH MM SS in each byte
18	Tenths of a second
19	Display mode: bits 2-0: 000=time 001=Up Timer 010=Down Timer 011=Interval Count Up 100=Interval Count Down bit5 (0x20): display mode 1=D:H:M mode (bit7 cleared) bit6 (0x40): start/stop 0=stopped 1=running bit7 (0x80): display mode 0=H:M:S 1=M:S:Tenths



20	Downtimer Alarm Set Bit0-6: Alarm Duration Bit7: Down timer Alarm Checked 0=unchecked 1=checked
21	Days currently displayed, most significant 8 bits
22	Digits value. 0=4/6 Digits, 1=(D):H:M:S, 2=(H):M:S.Tenths Options 1 and 2 require 9 digit display for () value to show Top 3 bits are the most significant bits of Day value
23	WiFi Signal Strength. 0 for wired, otherwise negative of value in dBm
24 to 39	Device Name as null terminated ASCII string

TM-Manager / TM-Timer uses this protocol to find and monitor clocks on the network. A Wireshark capture of that software can be used to see an example of the data transfer.

1.1.2 TM1000/TM2000 Device Query

Requesting information from the TM1000A/TM2000A is done by sending a 3 byte message to the TMX000A, using UDP/IP, to port 7372. The three bytes, in hexadecimal, are: 0xA1 0x04 0xB2 The TMX000A will also respond to a broadcast to the same port.

The response packet is 80 bytes and will be formatted as follows:

Bytes	Description
0	TM1000A response value = 0x04, TM2000A=0x05
1 to 4	client IP address
5 to 10	MAC address
11 and 12	firmware version Major:Minor
13	Lock status 0=No Lock, 1=2D Lock, or 2=3D Lock
14 to 17	NTP Sync count, 32 bits, MSB to LSB
18 to 20	Current Time, H:M:S, UTC
21 to 45	Location of unit 25 bytes, Latitude, Longitude, null terminated
46 to 80	Name of Time Server, null terminated

TM-Manager / TM-Timer uses this protocol to find and monitor TM1000A/TM2000A's on the network. A Wireshark capture of that software can be used to see an example of the data transfer.



1.2 Timer Control Sequences

The Up/Down counter timers can be controlled use the same UDP/IP API. Those sequences are documented in the following sections.

1.2.1 Use UpTimer

This command puts the clock into the UpTimer Mode. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xA2
1	Timer Display Mode: 0x00=MIN:SEC.Tenths, 0x01=HH:MM:SS
2	0x00

1.2.2 UpTimer Start/Pause

Toggles the UpTimer between running and paused. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xA3
1	0x00=Pause 0x01=Count Up **New in version 1.1
2	0x00

1.2.3 Uptimer Reset

Resets the UpTimer back to zero. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xA4
1	Timer Display Mode: 0x00=MIN:SEC.Tenths, 0x01=HH:MM:SS
2	0x00

1.2.4 Use DownTimer

Sets DownTimer Mode on the clock. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xA5
1	Timer Display Mode: 0x00=MIN:SEC.Tenths, 0x01=HH:MM:SS
2	Starting Hour value for countdown
3	Starting Minute value for countdown
4	Starting Second value for countdown
5	Starting Tenths of a second value for countdown
6	End of countdown Alarm Enable. 0=Disabled, 1=Enabled
7	Alarm duration in seconds
8	Starting Days value for countdown, LSB (optional, zero if omitted)
9	Starting Days value for countdown, MSB (optional, zero if omitted)



1.2.5 DownTimer Start/Pause

Toggles the DownTimer between running and paused. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xA6
1	0x00=Pause 0x01=Count Up **New in version 1.1
2	0x00

1.2.6 DownTimer Reset

Resets the DownTimer back to starting value. Must already be in DownTimer Mode. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xA7
1	Timer Display Mode: 0x00=MIN:SEC.Tenths, 0x01=HH:MM:SS
2	Starting Hour value for countdown
3	Starting Minute value for countdown
4	Starting Second value for countdown
5	Starting Tenths of a second value for countdown
6	End of countdown Alarm Enable. 0=Disabled, 1=Enabled
7	Alarm duration in seconds
8	Starting Days value for countdown, LSB (optional, zero if omitted)
9	Starting Days value for countdown, MSB (optional, zero if omitted)

1.2.7 Set Clock to TimeMode

Returns the clock from the Up/Down Timer modes to regular time display. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xA8
1	0x01
2	0x00

1.2.8 Set DotMatrix Text

Sets the text string on the DotMatrix display. Scrolling direction and Justification are controlled as well. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xA9
1	ESC Character for Formatting: 0x1B
2	Bits 3:2=Scroll Direction (0=No Scroll, 1=Right to Left, 2=Bottom to Top Bits 1:0=Justification of Text (1=Left, 2=Center, 3=Right)
3	Scroll Speed
4	Text to display starts here, null terminated. 250 chars maximum with null



1.2.9 Set UpTimer Time While Running

Instantaneously changes the time of the UpTimer while it is running. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xAA
1	New HOUR value
2	New MINUTE value
3	New SECOND value
4	New TENTHS of a second value
5	New HUNDREDTHS of a second value
6	New DAYS value, LSB (optional, if not present zero will be assumed)
7	New DAYS value, MSB (optional, if not present zero will be assumed)

1.2.10 Set DownTimer Time While Running

Instantaneously changes the time of the DownTimer while it is running. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xAB
1	New HOUR value
2	New MINUTE value
3	New SECOND value
4	New TENTHS of a second value
5	New HUNDREDTHS of a second value
6	New DAYS value, LSB (optional, if not present zero will be assumed)
7	New DAYS value, MSB (optional, if not present zero will be assumed)



1.2.11 Countdown to Date Timer Reset

Resets the Date Timer back to starting value. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xB9
1	Display Mode: 0x00=(HR):MIN:SEC.Tenths 0x01=(DAY):HH:MM:SS 0x02=DD:HH:MM
2, 3	Ending Year value for count down to date (ie. 2022). LSB, MSB
4	Ending Month value for count down to date
5	Ending Day value for count down to date
6	Ending Hour for count down to date
7	Ending Minute for count down to date
8	Ending Second for count down to date
9	End of countdown Alarm Enable. 0=Disabled, 1=Enabled
10	Alarm duration in seconds

1.2.12 Countdown to Date Start/Pause

Toggles the DownTimer between running and paused. A single character 'A' Acknowledge is sent back.

Bytes	Description
0	Command Byte: 0xBA
1	0x00=Pause 0x01=Count Up
2	0x00

1.2.13 Execute Stored Program

Tells the clock to load a particular stored timer program from the flash memory, and begin executing it. This is the same mechanism used by the Alarm system in the displays to trigger a stored program at a given time.

Bytes	Description
0	Command Byte: 0xB8
1	Program Number: 0-9



1.3 Misc Control Sequences

1.3.1 Relay Close

Immediately close the relay on the display for X seconds.

Bytes	Description
0	Command Byte: 0xB4
1	Number of seconds to close
2	0x00

1.3.2 Relay Toggle

Set the relay to either the closed or open position indefinitely. This condition is not checked by other events which could change the position of the relay. For example, if a timer ended with a triggered relay closure, that would change the state of the relay independent of this command.

Bytes	Description
0	Command Byte: 0xBC
1	0=open, 1=closed
2	0x00

1.3.3 Revert to Time Timeout

Prior to this release of the API, there was a hard-coded 30 minute timeout that would revert the clock from a non-running, un-used timer back to time display on the clock. This command allows this value to be changed. A value of 0, disables the revert timer. This value is not saved to non-volatile memory and will need to be updated any time power is lost.

Bytes	Description
0	Command Bytes: 0xBB
1	Timeout in Minutes, LSB
2	Timeout in Minutes, MSB

1.3.4 Dimmer Set

Immediately change the dimming level of the display. This setting is not saved to non-volatile memory and is thus only active until changed or the clock reboots.

Bytes	Description
0	Command Byte: 0xB5
1	Digit Brightness: 0-100
2	AM/PM/Colon dot brightness: 0-100



1.3.5 Color Set

On RGB equipped displays, immediately change the colors being displayed. This setting is not saved to non-volatile memory and is thus only active until changed or the clock reboots.

Bytes	Description
0	Command Byte: 0xB6
1, 2, 3	MM:SS Digit Color RGB components: 0-255
4, 5, 6	HH Digit Color RGB components: 0-255





2 Revision History

2.1 Version 1.0

Initial Release Feb 9, 2018

2.2 Version 1.1

Updated March 22, 2018

- UpTimer and DownTimer Start/Pause commands were updated such that the second byte of each now has meaning in the command. This prevented repeated packets from negating the previous packets meaning. Support for this started in version 2.3 of the WiFi clock, 4.5 of the POE clock, and 1.1 of the DotMatrix.
- Support for TM2000A locator protocol.

2.3 Version 2.0

Updated March 2021

- Changed format of general Locator response for clocks to allow displays to report timer function in real time
- Added direct relay control, dimmer control, RGB color set

2.4 Version 2.1

- Added support for countdown to date
- Added support for 9 digit clock displays
- Added option to toggle relay on and off